

Direcionamento de Infraestrutura PoC K6 Performance

Documento preparado pela Foursys para apresentação ao cliente Zurich.
Versão: v2 | Data: março 2026

1. Visao Geral da Integracao

O K6 se integra a três pontos do ecossistema Zurich: Azure DevOps (orquestracao), SonarQube (quality gate de código) e a Aplicação Piloto (alvo dos testes). Cada ponto de integração tem um papel específico e complementar.

O objetivo não é substituir nenhuma ferramenta existente. O K6 adiciona uma trilha de performance ao que já existe, sem alterar a esteira corporativa.

2. Integracao K6 com Azure DevOps

2.1. Como explicar para o cliente

O K6 vira um stage dentro da pipeline que vocês já usam. Não é uma ferramenta solta. Ele roda como parte do fluxo que vocês já conhecem no Azure DevOps.

2.2. Como funciona na pratica

O pipeline do Azure DevOps ganha um novo stage chamado 'Performance Test'. Esse stage só executa depois que o Build e o Quality Gate do SonarQube passarem com sucesso.

Exemplo de pipeline YAML com o stage de K6:

```
stages:
- stage: Build
  jobs:
  - job: BuildApp
    steps:
    - script: dotnet build

- stage: QualityGate
  dependsOn: Build
  jobs:
  - job: SonarAnalysis
    steps:
    - script: sonar-scanner analyze

- stage: PerformanceTest
  dependsOn: QualityGate
  condition: succeeded()
  jobs:
  - job: RunK6
    pool: 'SelfHostedPool-QA'
```

```
steps:
- task: AzureKeyVault@2
  inputs:
    azureSubscription: 'zurich-sub'
    KeyVaultName: 'kv-qa-zurich'
    SecretsFilter: 'api-token,api-url'

- script: |
  k6 run scripts/load-test.js \
    --env API_URL=$(api-url) \
    --env API_TOKEN=$(api-token) \
    --out json=results/output.json
  displayName: 'Executar K6'

- task: PublishBuildArtifacts@1
  inputs:
    PathToPublish: 'results/'
    ArtifactName: 'k6-results'
```

2.3 Pontos-chave

- O K6 só roda depois que o build e o quality gate passam
- O disparo pode ser automático (a cada push ou merge) ou manual (Run Pipeline no portal)
- Os segredos vem do Key Vault, nunca ficam no código
- Os resultados são publicados como artefato da pipeline, acessíveis no portal do Azure DevOps
- Se os thresholds falharem, o stage fica vermelho e o pipeline para

2.4 Modos de execucao

Modo	Como dispara	Quando usar
Automa-tico	Pipeline roda a cada push/merge na branch con-figurada	Regressao continua, quality gate automatico
Manual	Run Pipeline no portal do Azure DevOps ou CLI	Testes sob demanda, validacao antes de rele-ase
Agendado	Cron trigger na pipeline (ex: toda noite)	Testes de carga periodicos, baseline de per-formance

3. Integração K6 com SonarQube

3.1 Como explicar para o cliente

O SonarQube continua fazendo o que já faz: análise de código e quality gate de qualidade. O K6 adiciona um segundo quality gate, focado em performance. Os dois são complementares.

3.2 Dois quality gates sequenciais

O SonarQube e o K6 não conversam diretamente entre si. Eles operam como dois gates sequenciais dentro da mesma pipeline:

Build --> SonarQube (quality gate de código) --> K6 (quality gate de performance)

- Se o SonarQube reprovar (ex: bugs críticos, cobertura baixa), o K6 nem roda
- Se o SonarQube aprovar mas o K6 reprovar (ex: p95 acima de 800ms), o pipeline falha no stage de performance
- Se ambos aprovarem, o pipeline segue para o próximo passo (deploy, evidências, etc.)

3.3 Como configurar thresholds no K6

O quality gate de performance se define dentro do script K6:

```
export const options = {
  thresholds: {
    http_req_duration: ['p(95)<800'], // 95% das requests < 800ms
    http_req_failed: ['rate<0.02'], // menos de 2% de erro
    http_reqs: ['rate>50'], // pelo menos 50 req/s
  },
  stages: [
    { duration: '1m', target: 20 }, // ramp-up
    { duration: '3m', target: 50 }, // carga sustentada
    { duration: '1m', target: 0 }, // ramp-down
  ],
};
```

Se qualquer threshold for violado, o K6 retorna exit code diferente de zero e o Azure DevOps marca o stage como failed.

3.4 Comparação SonarQube vs K6

Aspecto	SonarQube	K6
O que analisa	Código-fonte (bugs, vulnerabilidades, cobertura)	Comportamento real da aplicação sob carga
Quando roda	Depois do build	Depois do SonarQube
Tipo de gate	Qualidade de código	Performance e estabilidade
Métricas	Bugs, code smells, cobertura, duplicação	Tempo de resposta, throughput, taxa de erro, p95/p99
Resultado	Aprovado/Reprovado no portal SonarQube	Aprovado/Reprovado no pipeline (thresholds)

4. Integração K6 com a Aplicação Zurich

4.1 Como explicar para o cliente

O K6 simula usuários reais acessando as APIs e endpoints da aplicação de vocês. Ele não instala nada na aplicação. Ele apenas gera tráfego controlado de fora para dentro, exatamente como um usuário faria, mas em escala.

4.2 Fluxo de rede

O K6 Executor (a VM) se conecta a aplicação-alvo pela rede privada (VNet/Subnet/NSG):

VM K6 Executor --> VNet/Subnet --> NSG (regras de acesso) --> Aplicação Zurich (APIs)

4.3 Exemplo de script K6

O script faz chamadas HTTP reais contra a aplicação piloto:

```
import http from 'k6/http';
import { check, sleep } from 'k6';

const BASE_URL = __ENV.API_URL;
const TOKEN = __ENV.API_TOKEN;

export default function () {
  const headers = {
    Authorization: `Bearer ${TOKEN}`,
    'Content-Type': 'application/json',
  };

  const res = http.get(`${BASE_URL}/api/v1/policies`, { headers });

  check(res, {
    'status 200': (r) => r.status === 200,
    'tempo < 500ms': (r) => r.timings.duration < 500,
  });

  sleep(1);
}
```

4.4 Pontos-chave

- Nada é instalado na aplicação: o K6 é um gerador de tráfego externo
- A conexão é privada: passa pela VNet, sem necessidade de expor APIs publicamente
- A autenticação é real: usa os mesmos tokens/credenciais que um usuário usaria
- O tráfego é controlado: o script define quantos usuários virtuais, por quanto tempo, com ramp-up gradual
- Os resultados medem a aplicação real: tempo de resposta, taxa de erro, throughput, p95, p99

5. Visão Consolidada do Fluxo

Fluxo completo da integração, do disparo até os resultados:

1. Time QA cria/atualiza scripts K6 no repositório
2. Pipeline é disparada (automática ou manual) no Azure DevOps
3. Stage Build compila a aplicação
4. Stage SonarQube executa análise de código e quality gate
5. Se SonarQube aprova, Stage PerformanceTest inicia
6. Self-hosted Agent (Runner) recebe o job
7. Runner busca segredos no Key Vault
8. Runner aciona o K6 na VM Executor
9. K6 gera carga contra a Aplicação Piloto Zurich (via VNet privada)
10. K6 valida thresholds (p95, erro, throughput)
11. Resultados são publicados no Storage e como artefato da pipeline
12. Azure Monitor e Log Analytics capturam telemetria
13. Dashboards e alertas dão visibilidade operacional
14. Se thresholds falharem, pipeline para e alerta o time

6. VMs Necessárias na Infraestrutura

Considerando que o Runner é reaproveitado da PoC do Selenium Grid, a trilha de K6 adiciona apenas 1 VM dedicada ao total:

VM	Função	Compartilhada?
VM 1	Self-hosted Agent / Runner (funcional + performance)	Sim, atende ambas as trilhas
VM 2	Selenium Grid Hub	Dedicada para funcional
VM 3 (VMSS)	Browser Nodes (Scale Set)	Dedicada para funcional
VM 4	K6 Executor - carga e estresse	Dedicada para performance

Total: 4 VMs para a plataforma completa (Selenium Grid + K6).

7. Perguntas Frequentes do Cliente

Pergunta	Resposta
Preciso instalar algo na minha aplicação?	Não. O K6 gera trafego de fora. A aplicação não precisa de nenhuma mudança.
Isso vai derrubar meu ambiente?	Na PoC, a carga e pequena e controlada. O script define limites e ramp-up gradual.
O SonarQube já não cobre isso?	O SonarQube analisa qualidade de código. O K6 testa comportamento real sob carga. São complementares.
Posso rodar sem pipeline, manualmente?	Sim. O K6 pode ser executado por SSH na VM ou por Run Pipeline manual no portal.
Os resultados ficam onde?	Na pipeline como artefato, no Storage Account e opcionalmente em dashboards do Azure Monitor.
Depois da PoC, meu time consegue rodar sozinho?	Sim. Entregamos o ambiente, os scripts-modelo e a documentação para autonomia.
Posso começar mais simples?	Sim. Na PoC começamos com 1 VM, poucos cenários e APIs piloto. A escala vem depois.
Como isso escala depois?	Adicionando mais VMs de executor, mais cenários e transformando thresholds em quality gate obrigatório.
Precisa de AKS?	Não. A solução roda em VMs simples, sem dependência de Kubernetes.
Qual a diferença entre K6 e Selenium	Selenium valida comportamento funcional (UI, jornadas). K6 valida performance (carga, resposta, estabilidade).